

90-DAY STUDY PLAN · FREE RESOURCES ONLY

Job-Ready AI Engineer

A complete week-by-week roadmap from Python foundations to deployed RAG systems and AI agents, with every resource linked.

October 1 - December 31, 2026

14

weekly plans

3

portfolio projects

385+

focused hours

₹0

cost of resources

Learn · Build · Evaluate · Ship

Resource availability and free-tier limits change. Verify links and limits before relying on them.

Contents

1. Read This First	3
2. What "Job-Ready AI Engineer" Means	4
3. Roadmap at a Glance	5
4. Tools, Free Compute and Free APIs	6
5. Weekly Plan: Weeks 0 – 13	7
Week 0: Launch: Setup, Baseline and Python Warm-up (Oct 1 – Oct 4)	8
Week 1: Python and Software Engineering for AI (Oct 5 – Oct 11)	10
Week 2: Data Skills and Math for Machine Learning (Oct 12 – Oct 18)	12
Week 3: Machine Learning I: Supervised Learning (Oct 19 – Oct 25)	14
Week 4: Machine Learning II and Project 1 (Oct 26 – Nov 1)	16
Week 5: Deep Learning with PyTorch (Nov 2 – Nov 8)	18
Week 6: Transformers and LLMs Under the Hood (Nov 9 – Nov 15)	20
Week 7: LLM APIs, Prompt and Context Engineering (Nov 16 – Nov 22)	22
Week 8: RAG I: Retrieval-Augmented Generation (Nov 23 – Nov 29)	24
Week 9: RAG II: Evaluation, Observability and Project 2 (Nov 30 – Dec 6)	26
Week 10: Agents I: Tool Use, Planning and LangGraph (Dec 7 – Dec 13)	28
Week 11: Agents II: MCP, Safety and Capstone Build (Dec 14 – Dec 20)	30
Week 12: Production: Deployment, LLMOps and Fine-tuning (Dec 21 – Dec 27)	32
Week 13: Final Sprint: Portfolio, Resume and Interviews (Dec 28 – Dec 31)	34
6. Portfolio and Capstone Projects	36
7. Interview Preparation	37
8. Portfolio, Résumé and Job Hunt	39
9. Master Resource Index	40
10. Progress Tracker	44
11. Self-Assessment Rubric	45
12. If You Fall Behind (or Get Ahead)	46

1. Read This First

The mission. By December 31, 2026 you will have **three deployed projects** (a machine-learning service, a production-style RAG assistant with an evaluation report, and an agentic capstone built on MCP), working knowledge of ML, deep learning and LLM fundamentals, and an interview-ready story about everything you built. Every resource in this guide is free.

The philosophy. Companies hiring AI engineers in 2026 mostly build products on top of existing models. The repeated asks in job descriptions are Python and software engineering, LLM APIs, retrieval (RAG), agents and tool use, evaluation, and deployment. This plan teaches enough ML and deep learning to be credible in interviews and to debug real systems, then spends the majority of your time on the application layer, where junior AI engineers are actually hired.

ASSUMPTIONS ABOUT YOUR STARTING POINT

- You can write basic Python and have some programming experience (web, scripting or coursework). Week 1 fills gaps.
- No formal ML background is required. Maths is rebuilt from intuition in Week 2.
- A laptop and internet. No GPU needed: you will use free Colab and Kaggle GPUs and free LLM API tiers.
- About 30 focused hours per week (Weeks 1–12). If you have less, follow the compressed path in Section 12.

WEEKLY RHYTHM (≈ 30 HOURS)

DAY	HOURS	WHAT TO DO
Mon – Fri	≈ 4 / day	2 h learn (course, docs, video) + 1.5 h build (practice tasks) + 0.5 h DSA or revision. Always code the same day you learn.
Saturday	5 – 6 h	Project day: build the week deliverable, write the README, push to GitHub.
Sunday	≈ 3 h	Retro (20 min), notes, 4 DSA problems, set next week's three outcomes. Finish the deliverable if behind. Keep part of Sunday free.

SEVEN RULES THAT MAKE THIS WORK

1. **Build more than you watch.** Aim for 70% hands-on. If you finish a video without writing code, you have not finished the video.
2. **Ship something every week.** Every week ends with a pushed repo or notebook. Your GitHub is your résumé.
3. **Stay on one stack.** Python, FastAPI, PyTorch, Hugging Face, plain Python or LlamaIndex for RAG, LangGraph for agents. Resist framework hopping.
4. **Measure everything.** If you cannot say how you know it works, you do not know it works. Evals start in Week 7, not at the end.
5. **Apply the 30-minute rule.** Stuck for 30 minutes? Read the error, search, ask. Write down what fixed it in your notes.
6. **Teach what you learn.** A short blog post or README explanation exposes gaps faster than rereading.
7. **Protect your energy.** Sleep, movement and one real day off every two weeks are part of the plan, not a reward for finishing it.

Priority tags. Every resource is tagged **CORE** (do it) or **OPTIONAL** (use if you have time or need a second explanation). If a week is overloaded, drop OPTIONAL items first, never the deliverable.

2. What "Job-Ready AI Engineer" Means

An **AI engineer** designs, builds, evaluates and ships software that uses machine-learning models, most often large language models, inside real products. The role sits between software engineering and applied ML. Hiring teams usually care about three things: can you **build** it, can you show it **works** (evidence, not opinion), and can you **ship and operate** it.

HOW THE ROLE COMPARES

ROLE	MAIN FOCUS	TYPICAL WORK	COVERED HERE
AI Engineer	Applications on top of models	LLM features, RAG, agents, evals, APIs, deployment	Primary target (Weeks 7–13)
ML Engineer	Training and serving models	Pipelines, training, feature stores, MLOps	Foundations (Weeks 3–5, 12)
Data Scientist	Analysis and modelling	EDA, statistics, experimentation, dashboards	Foundations (Weeks 2–4)
ML Researcher	New methods and models	Papers, large-scale training, novel architectures	Not the target; Week 6 gives the base

TEN OUTCOMES YOU WILL OWN BY DECEMBER 31

#	OUTCOME	JOB-READY BAR
1	Software craft	Write typed, tested, version-controlled Python and ship an API with FastAPI.
2	Data fluency	Clean, explore and query data with NumPy, Pandas and SQL.
3	ML fundamentals	Train, validate and evaluate classical models without leakage; choose metrics correctly.
4	Deep learning	Train a neural network in PyTorch and explain backpropagation.
5	LLM internals	Explain tokenisation, attention, pretraining and sampling, and debug LLM behaviour.
6	LLM applications	Build reliable features with structured outputs, tool calling, streaming and tested prompts.
7	Retrieval (RAG)	Build a hybrid-search RAG pipeline with reranking and citations over a real corpus.
8	Evaluation	Create golden sets, run retrieval and generation metrics, trace failures and prove improvements.
9	Agents and MCP	Build agents with tools, memory, guardrails and a custom MCP server; test them.
10	Production and communication	Dockerise, add CI, deploy on free tiers, and present your work clearly in a README, demo and interview.

Be realistic about hiring. Twelve weeks builds strong, demonstrable foundations and a portfolio. Offers also depend on the market, referrals and interview practice. Start applying in **Week 9** (Nov 30) once Project 2 is live, and keep your DSA and CS fundamentals warm, because many fresher roles blend software engineering with AI.

3. Roadmap at a Glance



PHASE	WEEKS	DATES	OUTCOME
0 • Launch	Week 0	Oct 1 – 4	Environment, accounts, free API keys, repo, baseline scores.
1 • Foundations	Weeks 1 – 4	Oct 5 – Nov 1	Python engineering, data and maths, classical ML. Project 1: ML service.
2 • Deep learning and LLM core	Weeks 5 – 6	Nov 2 – 15	PyTorch, backprop, transformers, mini-GPT, tokenisation.
3 • Applied AI engineering	Weeks 7 – 9	Nov 16 – Dec 6	LLM APIs, RAG, evaluation, observability. Project 2: RAG assistant with evals.
4 • Agents and production	Weeks 10 – 12	Dec 7 – 27	Agents, MCP, safety, deployment, CI, fine-tuning. Project 3: agentic capstone.
5 • Launch	Week 13	Dec 28 – 31	Portfolio, résumé, mock interviews, applications.

WEEK BY WEEK

WK	DATES	THEME	KEY DELIVERABLE
0	Oct 1 – Oct 4	Launch: Setup, Baseline and Python Warm-up	Repo, API keys, baseline scores
1	Oct 5 – Oct 11	Python and Software Engineering for AI	FastAPI service with tests; async caller
2	Oct 12 – Oct 18	Data Skills and Math for Machine Learning	EDA notebook; gradient descent from scratch
3	Oct 19 – Oct 25	Machine Learning I: Supervised Learning	Two Kaggle submissions; metrics cheat sheet
4	Oct 26 – Nov 1	Machine Learning II and Project 1	PROJECT 1: end-to-end ML service
5	Nov 2 – Nov 8	Deep Learning with PyTorch	micrograd + CIFAR-10 CNN
6	Nov 9 – Nov 15	Transformers and LLMs Under the Hood	Mini-GPT + blog post
7	Nov 16 – Nov 22	LLM APIs, Prompt and Context Engineering	llm-toolkit + three mini-apps
8	Nov 23 – Nov 29	RAG I: Retrieval-Augmented Generation	RAG pipeline with citations; golden set
9	Nov 30 – Dec 6	RAG II: Evaluation, Observability and Project 2	PROJECT 2: RAG assistant, eval report, live demo
10	Dec 7 – Dec 13	Agents I: Tool Use, Planning and LangGraph	Agent from scratch + LangGraph version
11	Dec 14 – Dec 20	Agents II: MCP, Safety and Capstone Build	Capstone MVP + MCP server + injection tests
12	Dec 21 – Dec 27	Production: Deployment, LLMOps and Fine-tuning	PROJECT 3: capstone deployed with CI, video
13	Dec 28 – Dec 31	Final Sprint: Portfolio, Resume and Interviews	Portfolio, résumé, 10 applications

4. Tools, Free Compute and Free APIs

Keep your tooling boring and consistent. The recommended stack below is what the weekly plans assume.

LAYER	RECOMMENDED TOOLS
Core	Python 3.12+, uv (or venv), VS Code, Jupyter, Git and GitHub, a terminal you are comfortable in
Engineering	FastAPI, Pydantic, pytest, httpx, Docker, GitHub Actions
Data and ML	NumPy, Pandas, Matplotlib / Seaborn, scikit-learn, XGBoost, MLflow
Deep learning	PyTorch, Hugging Face Transformers, PEFT, Colab and Kaggle GPUs
LLM apps	Gemini / Groq / OpenRouter / Ollama via a thin wrapper or LiteLLM; Pydantic for structured outputs
RAG	Sentence-Transformers, Chroma or FAISS (pgvector later), BM25, a cross-encoder reranker, LlamaIndex or plain Python
Agents	LangGraph, MCP Python SDK, smolagents for comparison
Evals and tracing	Ragas, promptfoo, Langfuse or Phoenix, your own pytest-based golden sets
Deploy	Hugging Face Spaces, Streamlit Community Cloud, Render, Google Cloud Run; Vercel for a static frontend

FREE COMPUTE AND APIS (WHAT YOU ACTUALLY GET)

RESOURCE	WHAT YOU GET	USE IT FOR	WATCH OUT FOR
Google Colab	Free notebook sessions, GPU when available	PyTorch, fine-tuning labs	Sessions time out; GPU availability varies
Kaggle Notebooks	Free GPU / TPU with a weekly quota	Training, competitions, datasets	Check the current weekly limit
Google AI Studio (Gemini API)	Free tier for Flash-class models; limits shown in the AI Studio console	Chat, structured output, tool calling, embeddings	Pro-class models and limits change; data policies differ on free tier
Groq	Free plan with per-model rate limits; very fast open models	Agents and fast iteration	Limits are per organisation and model
OpenRouter	Models tagged free, with daily request caps	Trying many open models	Low daily caps; availability changes
Ollama	Run open models on your own machine	Offline dev, privacy, zero cost	Pick small models your hardware can run
Hugging Face	Hub, datasets, free Spaces (CPU)	Demos, model downloads	Free Spaces sleep when idle
Langfuse / Phoenix	Open-source tracing; free cloud tier or self-host	Observability and evals	Read current free-tier limits

About paid-API providers. Anthropic does not publish a standing free API tier, and OpenAI's free usage depends on region and changes over time. You can still learn their documentation and courses for free (they are linked in Weeks 7, 10 and 11) and practise the same patterns on Gemini, Groq, OpenRouter or Ollama. Write provider-agnostic code (a thin wrapper or LiteLLM) so you can swap models without rewriting your project. Free-tier limits change often; verify them in each provider's console before you depend on them.

5. Weekly Plan: Weeks 0 – 13

Each weekly page has the same structure so you can move fast: **goal**, **what you will learn**, **free resources** with direct links, a **day-by-day plan**, **hands-on practice**, a **deliverable**, a self-check ("ready to move on when you can...") and a **retro box** for Sunday.

HOW TO USE A WEEKLY PAGE

1. On the first day, read the goal and the self-check. Decide what "done" looks like for this week.
2. Follow the day-by-day plan. Do CORE resources first; use OPTIONAL ones only when you need a second explanation.
3. Build the practice tasks the same day you learn the concept. Commit often.
4. Finish the deliverable by Saturday. Use Sunday for the retro, notes and interview practice.
5. If you cannot tick every item in the self-check, spend half of next Monday closing the gap before moving on.

PHASE	WEEKS	PROJECT MILESTONE
Foundations	1 – 4	Project 1 (ML service) ships at the end of Week 4
Deep learning and LLM core	5 – 6	No project; a mini-GPT and a blog post that prove understanding
Applied AI engineering	7 – 9	Project 2 (RAG assistant with evals) ships in Week 9. Begin applying.
Agents and production	10 – 12	Project 3 (agentic capstone) ships in Week 12
Launch	13	Portfolio, résumé, mock interviews and applications

WEEK
0

Launch: Setup, Baseline and Python Warm-up

Phase 0 · Launch

Oct 1 – Oct 4

~12 hrs

GOAL Get a working development environment, the right accounts, free API keys and a public GitHub repo for the whole journey. Measure your starting point honestly so you can see progress later.

Starting late? If you begin on or after Oct 3, merge Days 1–2 into one session and keep Monday Oct 5 as the firm start of Week 1.

WHAT YOU WILL LEARN

- Dev environment: Python 3.12+, uv or venv, VS Code, Jupyter, Git branches and pull requests
- Accounts: GitHub, Hugging Face, Kaggle, Google AI Studio, Groq (optional: OpenRouter)
- Calling a hosted LLM from Python with an API key stored in an environment variable
- Running a small open model locally with Ollama (pick a size your laptop can handle)
- Secrets hygiene: .env files, .gitignore, never committing keys
- Your study system: daily log, weekly retro, notes in the repo

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	GitHub: Start your journey Create the repo, learn commit/push/branch/PR flow. docs.github.com/en/get-started/start-your-journey
CORE	uv: fast Python project and package manager Create a project, add packages, lock dependencies. docs.astral.sh/uv
CORE	Python in Visual Studio Code Set up interpreter, Jupyter and debugging. code.visualstudio.com/docs/python/python-tutorial
CORE	Google AI Studio (free Gemini API key) Create a free API key; make your first call from Python. aistudio.google.com
CORE	Groq Console (free-tier API key) Second provider for comparison; same task, different model. console.groq.com
CORE	Ollama (run open models locally) Install and run one small model locally. ollama.com
OPTIONAL	The Missing Semester of Your CS Education (MIT) Watch the shell and Git lectures if the terminal feels slow. missing.csail.mit.edu
OPTIONAL	The Python Tutorial (official docs) Skim any Python topic you are unsure about. docs.python.org/3/tutorial

DAY-BY-DAY PLAN

WHEN	FOCUS
Day 1 • Thu	Install toolchain; create GitHub repo "ai-engineer-90"; write README with this plan's goals.
Day 2 • Fri	Get API keys (Gemini, Groq). Write hello_llm.py that reads the key from .env and prints a reply.
Day 3 • Sat	Install Ollama, run a small model, compare answers with the hosted model. Take the baseline self-assessment (Section 11).
Day 4 • Sun	Create the daily-log.md habit, plan Week 1 hour by hour, solve 3 easy DSA problems.

HANDS-ON PRACTICE

1. Create the repo structure: /notes, /week-01 ... /projects, daily-log.md.
2. hello_llm.py: send one prompt to two providers and print both answers plus response time.
3. Run one prompt on a local Ollama model and note differences in quality and speed.
4. Take the baseline rubric (Section 11) and commit your honest scores.

DELIVERABLE (push to GitHub)

- Public GitHub repo with README, daily-log.md and hello_llm.py
- API keys stored safely (not committed)
- Baseline rubric scores committed

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Create a virtual environment, install a package and run a script from the terminal.
- Call an LLM API from Python without hard-coding the key.
- Commit, push and open a pull request on GitHub.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / — target Energy (1-5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
1

Python and Software Engineering for AI

Phase 1 · Foundations

Oct 5 – Oct 11
~30 hrs

GOAL Write clean, typed, tested Python and ship a small HTTP API. This is the engineering base that separates an AI engineer from a notebook user.

WHAT YOU WILL LEARN

- Python depth: dataclasses, type hints, generators, decorators, context managers, exceptions
- Async I/O with asyncio and httpx; retries with exponential backoff; rate limits
- Data validation with Pydantic; JSON handling; configuration via environment variables
- Testing with pytest: fixtures, parametrization, mocking HTTP calls
- Logging, project layout, pyproject.toml, packaging basics
- REST fundamentals and FastAPI: routes, request/response models, auto docs
- Git workflow: small commits, branches, pull requests

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	CS50's Intro to Programming with Python (Harvard) Skip what you know; do the problem sets on functions, OOP, unit tests, file I/O. cs50.harvard.edu/python
CORE	asyncio (official Python docs) Tasks, gather, timeouts. Then write the async batch caller. docs.python.org/3/library/asyncio.html
CORE	Pydantic documentation Models, validators, settings management. docs.pydantic.dev/latest
CORE	pytest: getting started Fixtures and parametrize; use monkeypatch to mock network calls. docs.pytest.org/en/stable/getting-started.html
CORE	FastAPI tutorial Complete the tutorial through request bodies, dependencies and testing. fastapi.tiangolo.com/tutorial
OPTIONAL	HTTPX (async HTTP client) Async client, timeouts, retries. www.python-httpx.org
OPTIONAL	Real Python (free articles) Free articles on decorators, generators and typing. realpython.com
OPTIONAL	Pro Git (free book) Chapters 2–3 on branching if Git still feels shaky. git-scm.com/book/en/v2

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon–Tue	Python depth: typing, dataclasses, generators, decorators. Solve 8 small exercises.
Wed	asyncio + httpx: call a public API 20 times concurrently with retry and backoff.
Thu	Pydantic + pytest: validate messy JSON, write 10 tests including a mocked HTTP call.
Fri	FastAPI: build /health and /summarize (stub) with typed request and response models.
Sat	Build the week project and write the README. (5–6 hrs)
Sun	Retro, notes, 4 DSA problems (arrays and hashing), plan Week 2.

HANDS-ON PRACTICE

1. Build "textstats", a small CLI package: reads files, prints word counts and top terms. Typed, tested.
2. Write an async batch caller with retry, backoff and structured logging.
3. Build a FastAPI service with Pydantic models and tests using TestClient.
4. DSA: 4 problems on arrays and hash maps; write your solution pattern in notes.

DELIVERABLE (push to GitHub)

- week-01 repo: FastAPI service + tests passing + README with run instructions
- Async batch caller with backoff and logging

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Explain async vs threads vs processes in two minutes.
- Write a pytest that mocks an HTTP call and asserts on the result.
- Validate bad input with a Pydantic model and explain the error.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1–5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
2

Data Skills and Math for Machine Learning

Phase 1 · Foundations

Oct 12 – Oct 18
~30 hrs

GOAL Manipulate, query and visualise data fluently, and rebuild the maths intuition behind ML: vectors, gradients and probability.

WHAT YOU WILL LEARN

- NumPy: arrays, broadcasting, vectorisation. Pandas: groupby, joins, missing data, time series basics
- Visualisation with Matplotlib and Seaborn; an exploratory data analysis (EDA) workflow
- SQL: SELECT, JOINS, GROUP BY, window functions, CTEs
- Linear algebra: vectors, matrices, dot product, matrix multiplication, eigenvectors (intuition)
- Calculus: derivatives, chain rule, gradients, gradient descent
- Probability and statistics: distributions, expectation, variance, Bayes rule, hypothesis testing basics

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Kaggle Learn: Pandas Complete all lessons and exercises. www.kaggle.com/learn/pandas
CORE	Kaggle Learn: Data Visualization Complete; reproduce 3 chart types with your own data. www.kaggle.com/learn/data-visualization
CORE	Kaggle Learn: Intro to SQL Complete Intro to SQL, then start Advanced SQL (joins, analytic functions). www.kaggle.com/learn/intro-to-sql
CORE	3Blue1Brown: Essence of Linear Algebra Watch all episodes; pause and sketch the transformations. www.3blue1brown.com/topics/linear-algebra
CORE	3Blue1Brown: Essence of Calculus Watch derivatives, chain rule and optimisation episodes. www.3blue1brown.com/topics/calculus
CORE	StatQuest with Josh Starmer (YouTube) Watch statistics fundamentals: distributions, p-values, Bayes. www.youtube.com/@statquest
OPTIONAL	SQLBolt (interactive SQL lessons) 18 quick interactive SQL lessons. sqlbolt.com
OPTIONAL	LeetCode: Top SQL 50 study plan Do at least 20 of the 50 problems. leetcode.com/studyplan/top-sql-50
OPTIONAL	NumPy quickstart Reference while coding broadcasting exercises. numpy.org/doc/stable/user/quickstart.html
OPTIONAL	Khan Academy: Statistics and Probability Fill gaps in probability. www.khanacademy.org/math/statistics-probability
OPTIONAL	Mathematics for Machine Learning (free book) Read chapters 2 and 5 when you want the formal version. mml-book.github.io

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	NumPy + Pandas basics (Kaggle Pandas). Linear algebra videos 1–4.
Tue	Pandas advanced (groupby, merge) + visualisation. Linear algebra videos 5–8.
Wed	SQL: SELECT, JOIN, GROUP BY. 10 practice queries.
Thu	SQL: window functions and CTEs. Calculus videos on derivatives and the chain rule.
Fri	Probability and statistics: Bayes rule, variance, distributions.
Sat	EDA project on a real dataset; write the narrative. (5–6 hrs)
Sun	Retro, notes, 4 DSA problems (two pointers, sliding window), plan Week 3.

HANDS-ON PRACTICE

1. EDA notebook on a real Kaggle dataset: 10 written insights, 5 clean charts, clear narrative.
2. Solve 30 SQL problems (SQLBolt, Kaggle, LeetCode SQL 50).
3. From scratch in NumPy: matrix multiply, then gradient descent for linear regression; plot the loss curve.
4. Work three Bayes-rule examples by hand (medical test, spam filter, your own).

DELIVERABLE (push to GitHub)

- Clean EDA notebook with narrative
- Gradient-descent-from-scratch notebook
- SQL practice log (30 queries)

YOU ARE READY TO MOVE ON WHEN YOU CAN

- State the broadcasting rule and give an example.
- Explain what a gradient is and why we step against it.
- Apply Bayes rule to a base-rate problem and write a window-function query.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1–5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
3

Machine Learning I: Supervised Learning

Phase 1 · Foundations

Oct 19 – Oct 25
~30 hrs

GOAL Understand and apply regression and classification the right way, with clean validation and the correct metrics, using scikit-learn.

WHAT YOU WILL LEARN

- Linear and logistic regression, k-NN, decision trees; loss functions and gradient descent
- Train / validation / test splits, cross-validation, learning curves
- Overfitting vs underfitting, the bias-variance trade-off, L1/L2 regularisation
- Metrics: MSE/MAE, accuracy, precision, recall, F1, ROC-AUC, PR-AUC; class imbalance
- Data leakage: how it happens and how to prevent it
- scikit-learn API: fit / predict / transform, Pipelines

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Machine Learning Specialization, Andrew Ng (audit free) Course 1 (regression, classification). Skim what you already know; do the labs. www.coursera.org/specializations/machine-learning-introduction
CORE	Kaggle Learn: Intro to Machine Learning Complete; practise train/test splitting and model validation. www.kaggle.com/learn/intro-to-machine-learning
CORE	scikit-learn User Guide Read supervised learning and model selection sections for the models you use. scikit-learn.org/stable/user_guide.html
CORE	Google Machine Learning Crash Course Use as a second explanation for classification, metrics and data. developers.google.com/machine-learning/crash-course
CORE	Kaggle competition: Titanic Make a first submission this week. www.kaggle.com/c/titanic
CORE	Kaggle competition: House Prices Second submission using a regression pipeline. www.kaggle.com/c/house-prices-advanced-regression-techniques
OPTIONAL	Hands-On Machine Learning notebooks (Aurelien Geron) Open chapters 2–4 notebooks and read alongside your code. github.com/ageron/handson-ml3
OPTIONAL	Machine Learning Zoomcamp (DataTalks.Club) Modules 1–4 as an extra project-style course. github.com/DataTalksClub/machine-learning-zoomcamp
OPTIONAL	StatQuest with Josh Starmer (YouTube) ML playlist for logistic regression, regularisation, ROC. www.youtube.com/@statquest

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon–Tue	Ng Course 1: regression, gradient descent, feature scaling. Code linear regression from scratch.
Wed	Classification and logistic regression; metrics (confusion matrix, F1, ROC-AUC).
Thu	Kaggle Intro to ML + scikit-learn practice with train/test splits.
Fri	Overfitting, regularisation, cross-validation, leakage case studies.
Sat	Kaggle Titanic + House Prices notebooks. (5–6 hrs)
Sun	Write your metrics cheat sheet; 4 DSA problems (stacks, binary search); plan Week 4.

HANDS-ON PRACTICE

1. Implement linear and logistic regression from scratch in NumPy and compare against scikit-learn.
2. Submit to Kaggle Titanic and House Prices with a documented pipeline.
3. Write a one-page "which metric when" cheat sheet in your own words.
4. Find and describe one leakage bug in a public notebook.

DELIVERABLE (push to GitHub)

- Two Kaggle submissions with notebooks in the repo
- From-scratch regression notebook
- Metrics cheat sheet (markdown)

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Pick the right metric for an imbalanced classification problem and justify it.
- Give three ways to reduce overfitting.
- Explain why the test set is used only once and give a leakage example.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1–5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
4

Machine Learning II and Project 1

Phase 1 · Foundations

Oct 26 – Nov 1
~30 hrs

GOAL Build and ship a complete tabular ML project with ensembles, pipelines and experiment tracking. This is Project 1 in your portfolio.

WHAT YOU WILL LEARN

- Random forests and gradient boosting (XGBoost / LightGBM); hyperparameter search
- Feature engineering: encoding, scaling, imputation; ColumnTransformer + Pipeline
- Unsupervised learning: k-means, PCA; when clustering helps
- Interpretability: feature importance, a first look at SHAP
- Experiment tracking with MLflow; saving and loading models
- Serving a model behind FastAPI; writing a model card

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Kaggle Learn: Intermediate Machine Learning Pipelines, missing values, categorical variables, XGBoost, leakage. www.kaggle.com/learn/intermediate-machine-learning
CORE	Kaggle Learn: Feature Engineering Mutual information, creating features, target encoding. www.kaggle.com/learn/feature-engineering
CORE	Machine Learning Specialization, Andrew Ng (audit free) Courses 2–3: skim decision trees, ensembles and unsupervised learning. www.coursera.org/specializations/machine-learning-introduction
CORE	XGBoost documentation Work through the introductory tutorials and parameter notes. xgboost.readthedocs.io
CORE	MLflow documentation Tracking quickstart: log parameters, metrics and artifacts. mlflow.org/docs/latest
OPTIONAL	SHAP documentation Explain one model prediction with SHAP values. shap.readthedocs.io
OPTIONAL	Google's Rules of Machine Learning Read the first section for production mindset. developers.google.com/machine-learning/guides/rules-of-ml

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	Trees and random forests; compare against Week 3 baselines.
Tue	Gradient boosting with XGBoost; tune with cross-validation.
Wed	Pipelines and feature engineering; no leakage.
Thu	MLflow tracking; compare 3+ models in a results table.
Fri	Serve the best model with FastAPI (/predict) and add tests.
Sat	Finish Project 1: model card, README, architecture sketch. (5–6 hrs)
Sun	Retro; 4 DSA problems (linked lists, trees); plan Week 5.

HANDS-ON PRACTICE

1. Pick a real tabular dataset (churn, loan default, house price, fraud) and define the business question.
2. Compare at least 3 models with cross-validation inside a single Pipeline.
3. Track every experiment in MLflow; export a results table.
4. Expose /predict with FastAPI; add input validation and tests.

DELIVERABLE (push to GitHub)

- PROJECT 1: end-to-end ML service repo (EDA, pipeline, MLflow results, FastAPI /predict, tests)
- Model card: data, metrics, limitations, intended use

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Choose between random forest and boosting for a case and justify.
- Describe a pipeline that prevents leakage during cross-validation.
- Explain PCA and when you would not use it.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1-5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
5

Deep Learning with PyTorch

Phase 2 · Deep Learning and LLM Core

Nov 2 – Nov 8
~30 hrs**GOAL** Understand how neural networks learn, then train your own with PyTorch. Backprop should stop being a black box.

WHAT YOU WILL LEARN

- Tensors, autograd and backpropagation by hand
- MLPs, activation functions, loss functions, softmax and cross-entropy
- Optimisers (SGD, Adam), learning-rate schedules, initialisation
- Regularisation: dropout, weight decay, batch norm; data augmentation
- Training-loop discipline: overfit one batch, read loss curves, debug
- CNN basics and embeddings; using free GPUs on Colab / Kaggle; checkpoints

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Andrej Karpathy: Neural Networks, Zero to Hero Videos 1–3: micrograd, makemore bigram, makemore MLP. Code along. karpathy.ai/zero-to-hero.html
CORE	PyTorch: Learn the Basics Tensors, Datasets and DataLoaders, nn.Module, autograd, optimisation loop. pytorch.org/tutorials/beginner/basics/intro.html
CORE	3Blue1Brown: Neural Networks Watch the 4 episodes before coding backprop. www.3blue1brown.com/topics/neural-networks
CORE	fast.ai: Practical Deep Learning for Coders Lessons 1–3 for fast practical results and transfer learning. course.fast.ai
OPTIONAL	Dive into Deep Learning (free book) Look up chapters on MLPs, CNNs and optimisation as needed. d2l.ai
OPTIONAL	Karpathy: A Recipe for Training Neural Networks Karpathy checklist for debugging training. karpathy.github.io/2019/04/25/recipe
OPTIONAL	Stanford CS231n course notes Notes on optimisation and CNNs. cs231n.github.io

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	3Blue1Brown NN episodes + micrograd part 1 (code along).
Tue	Finish micrograd; PyTorch tensors and autograd.
Wed	PyTorch Dataset / DataLoader / nn.Module; first training loop on MNIST.
Thu	makemore: bigram then MLP; embeddings.
Fri	CNN on CIFAR-10 or Fashion-MNIST; add augmentation and dropout.
Sat	Improve the model, log curves, write the explanation. (5–6 hrs)
Sun	Retro; 4 DSA problems (heaps, recursion); plan Week 6.

HANDS-ON PRACTICE

1. Rebuild micrograd and train a tiny MLP on a toy dataset.
2. PyTorch CNN on CIFAR-10 reaching at least 75% test accuracy; record what helped most.
3. Deliberately break a model (bad LR, no normalisation) and fix it using the debugging recipe.
4. Write a one-page "How backpropagation works" in your own words.

DELIVERABLE (push to GitHub)

- micrograd notebook + CIFAR-10 / Fashion-MNIST notebook with curves
- One-page backprop explanation in the repo

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Walk through the chain rule on a two-layer network.
- Explain why Adam is popular and when SGD may generalise better.
- Debug a model that is not learning: list the first five checks.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1-5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
6

Transformers and LLMs Under the Hood

Phase 2 · Deep Learning and LLM Core

Nov 9 – Nov 15

~30 hrs

GOAL Understand exactly how modern LLMs work (tokenisation, attention, pretraining, post-training) well enough to explain and debug them.

WHAT YOU WILL LEARN

- Tokenisation (BPE), embeddings, positional information
- Self-attention, multi-head attention, the transformer block, decoder-only GPT
- Pretraining objective; scaling intuition; instruction tuning; RLHF and DPO (overview)
- Sampling: temperature, top-k, top-p; context window and KV cache; quantisation
- Open vs closed models; model cards; the Hugging Face Hub and Transformers library

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Andrej Karpathy: Neural Networks, Zero to Hero Videos "Let's build GPT" and "Let's build the GPT Tokenizer". Code along. karpathy.ai/zero-to-hero.html
CORE	Hugging Face LLM Course Chapters 1–4: Transformers, Datasets, Tokenizers, using models. huggingface.co/learn/llm-course
CORE	The Illustrated Transformer (Jay Alammar) Read first; it makes the code videos easier. jalammar.github.io/illustrated-transformer
CORE	Attention Is All You Need (paper) Read after the above; focus on sections 3.1–3.3. arxiv.org/abs/1706.03762
OPTIONAL	Build a Large Language Model From Scratch (code, Sebastian Raschka) Chapters 2–4 notebooks as a second implementation. github.com/rasbt/LLMs-from-scratch
OPTIONAL	The Illustrated GPT-2 (Jay Alammar) Visual tour of a decoder-only model. jalammar.github.io/illustrated-gpt2
OPTIONAL	Stanford CS224N: NLP with Deep Learning Lectures on transformers and pretraining. web.stanford.edu/class/cs224n

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	Illustrated Transformer; attention by hand on a tiny example.
Tue–Wed	Let's build GPT: code a character-level transformer end to end.
Thu	Tokenizer video; experiment with tiktoken / HF tokenizers; count tokens and cost.
Fri	HF Transformers: pipelines, generation parameters, run a small open model.
Sat	Write the blog post and run sampling experiments. (5–6 hrs)
Sun	Read the attention paper; retro; 4 DSA problems (graphs: BFS/DFS); plan Week 7.

HANDS-ON PRACTICE

1. Train a small GPT on a text corpus you choose; sample from it at different temperatures.
2. Use Hugging Face pipelines for three different tasks; inspect tokens and logits.
3. Measure token counts and cost estimates for 20 real documents.
4. Publish a blog post: "How an LLM generates text, explained with my own mini-GPT".

DELIVERABLE (push to GitHub)

- Mini-GPT notebook + sampling experiments
- Published blog post (GitHub Pages, Dev.to or Medium)

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Explain self-attention with queries, keys and values to a non-expert.
- Explain temperature and top-p, and what happens at temperature 0.
- Distinguish pretraining, supervised fine-tuning and RLHF.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1-5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
7

LLM APIs, Prompt and Context Engineering

Phase 3 · Applied AI Engineering

Nov 16 – Nov 22
~30 hrs

GOAL Build reliable LLM features: structured outputs, tool calling, streaming and tested prompts. This is where you start building like an AI engineer.

WHAT YOU WILL LEARN

- Messages API anatomy; system prompts, few-shot examples, when chain-of-thought helps
- Structured outputs with JSON schema and Pydantic; validation and repair loops
- Function / tool calling; streaming responses; multi-provider wrappers (LiteLLM)
- Reliability: timeouts, retries, rate limits, fallbacks, caching
- Cost, latency and token budgeting; context engineering (what belongs in the window)
- Prompt versioning and prompt unit tests; hallucination basics

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	DeepLearning.AI Short Courses (free) Take "ChatGPT Prompt Engineering for Developers" and one tool-use / function-calling short course. www.deeplearning.ai/short-courses
CORE	Anthropic courses (interactive prompting and tool-use tutorials) Interactive prompt-engineering and tool-use tutorials; do them with any model via your free API. github.com/anthropics/courses
CORE	Anthropic docs: Prompt engineering overview Read the techniques section: clarity, examples, structured prompts. docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview
CORE	Gemini API documentation Text generation, structured output, function calling on the free tier. ai.google.dev/gemini-api/docs
CORE	OpenAI Cookbook Pick examples on function calling and structured outputs; port them to your provider. cookbook.openai.com
OPTIONAL	Prompt Engineering Guide Reference for prompting techniques. www.promptingguide.ai
OPTIONAL	LiteLLM (one interface to many providers) Use one interface across providers. docs.litellm.ai
OPTIONAL	Anthropic Cookbook Tool-use and extraction recipes. github.com/anthropics/anthropic-cookbook
OPTIONAL	Simon Willison's blog (practical LLM engineering) Read posts on LLM tooling and prompt injection. simonwillison.net

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	API anatomy; prompting patterns; run the same task across two providers.
Tue	Structured outputs with Pydantic; build the extractor.
Wed	Tool / function calling; build the tool-using assistant.
Thu	Streaming, retries, caching; measure latency and token use.
Fri	Prompt unit tests: 20 golden cases with pytest.
Sat	Finish the three apps and the provider comparison. (5–6 hrs)
Sun	Retro; 4 DSA problems (binary search, intervals); plan Week 8.

HANDS-ON PRACTICE

1. Extractor: unstructured resume or invoice text into validated JSON with a retry-on-invalid loop.
2. Classifier: support-ticket categoriser evaluated on 50 labelled samples; report accuracy and confusion matrix.
3. Tool-calling assistant with at least 3 tools and a max-steps guard.
4. Compare two providers on 30 inputs for quality, latency and cost; write the table.

DELIVERABLE (push to GitHub)

- llm-toolkit repo: provider wrapper, retries, caching, structured-output helpers, 20 prompt tests
- Three working mini-apps + provider comparison table

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Explain few-shot vs chain-of-thought and when each hurts.
- Describe how tool calling works end to end, including who executes the tool.
- List four ways to cut cost and latency without losing quality.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1–5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
8

RAG I: Retrieval-Augmented Generation

Phase 3 · Applied AI Engineering

Nov 23 – Nov 29

~30 hrs

GOAL Build a retrieval pipeline first from scratch, then with a framework, over a real document set that you care about.

WHAT YOU WILL LEARN

- Why RAG; embeddings and similarity (cosine vs dot product); choosing an embedding model
- Loading and parsing PDFs, HTML and Markdown; cleaning text
- Chunking: size, overlap, structure-aware and semantic chunking; metadata
- Vector stores: Chroma, FAISS, pgvector; metadata filtering
- Keyword search (BM25) + dense retrieval = hybrid search; reranking with cross-encoders
- Query rewriting; prompt assembly with citations; contextual retrieval idea
- Framework vs from-scratch trade-offs (LlamaIndex, LangChain)

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	DeepLearning.AI Short Courses (free) Take "Building and Evaluating Advanced RAG Applications" and "Building Agentic RAG with LlamaIndex". www.deeplearning.ai/short-courses
CORE	LLM Zoomcamp (DataTalks.Club) RAG and vector-search modules; follow the code. github.com/DataTalksClub/llm-zoomcamp
CORE	Chroma documentation Collections, metadata, querying. docs.trychroma.com
CORE	Sentence-Transformers documentation Embeddings, semantic search and cross-encoder reranking. www.sbert.net
CORE	LlamaIndex documentation Starter tutorial, then the retrieval and query-engine sections. docs.llamaindex.ai
OPTIONAL	Pinecone Learning Center (RAG, chunking, reranking) Articles on chunking strategies, hybrid search and reranking. www.pinecone.io/learn
OPTIONAL	Anthropic: Introducing Contextual Retrieval Read the contextual retrieval idea; try it on your corpus. www.anthropic.com/news/contextual-retrieval
OPTIONAL	MTEB embedding leaderboard Compare embedding models for your language and task. huggingface.co/spaces/mteb/leaderboard
OPTIONAL	FAISS (vector similarity search library) Understand index types. github.com/facebookresearch/faiss
OPTIONAL	pgvector (vector search in Postgres) Vector search in Postgres. github.com/pgvector/pgvector

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	Embeddings and cosine similarity by hand; pick your corpus.
Tue	Loaders and chunking experiments; store chunks with metadata.
Wed	Naive RAG from scratch: embed, store, retrieve, answer with citations.
Thu	Hybrid search (BM25 + dense) and reranking.
Fri	Rebuild with LlamaIndex; compare effort and control.
Sat	Scale to 50+ documents; write ingestion script and CLI. (5–6 hrs)
Sun	Write a 30-question golden set with source passages for Week 9; retro; 4 DSA problems.

HANDS-ON PRACTICE

1. Choose a real corpus (library docs, a syllabus, public policy documents, a product handbook), not a toy dataset.
2. Build ingestion: parse, clean, chunk, embed, store with metadata.
3. Implement naive RAG, then add hybrid search and reranking; compare hit rate on 30 questions manually.
4. Answer with inline citations that point to chunk and source.

DELIVERABLE (push to GitHub)

- RAG repo: ingestion script, retrieval module, CLI that answers with citations
- Golden set: 30 questions with expected source passages

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Explain how chunk size affects retrieval and answer quality.
- Say when hybrid search beats dense-only and what reranking adds.
- Show how your system cites sources and what happens when nothing relevant is found.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1–5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
9

RAG II: Evaluation, Observability and Project 2

Phase 3 · Applied AI Engineering

Nov 30 – Dec 6
~30 hrs

GOAL Measure and improve quality systematically, then deploy. Evaluation skills are the strongest hiring signal for junior AI engineers. Start applying to roles this week.

WHAT YOU WILL LEARN

- Eval-driven development: golden datasets, one change at a time, regression tests
- Retrieval metrics: hit rate / recall@k, MRR, nDCG
- Generation metrics: faithfulness (groundedness), answer relevance, correctness
- LLM-as-judge: calibration, bias, and when to use human review
- Error analysis: read traces, categorise failures, fix the biggest bucket first
- Tracing and observability (Langfuse, Phoenix); cost and latency tracking
- Demo deployment on Streamlit Community Cloud or Hugging Face Spaces

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Hamel Husain: blog on LLM evals Read his posts on evals and error analysis before building your harness. hamel.dev/blog
CORE	Ragas documentation Compute faithfulness, answer relevance and context metrics. docs.ragas.io
CORE	Langfuse documentation (tracing and evals) Add tracing to your pipeline; inspect every call. langfuse.com/docs
CORE	promptfoo documentation (evals and red-teaming) Run regression-style evals from a config file. www.promptfoo.dev/docs/intro
CORE	Streamlit documentation Build the UI and deploy to Community Cloud (or use HF Spaces). docs.streamlit.io
CORE	Hugging Face Spaces documentation Alternative free deployment target. huggingface.co/docs/hub/spaces
OPTIONAL	Arize Phoenix documentation Open-source tracing and eval UI. docs.arize.com/phoenix
OPTIONAL	Eugene Yan: Patterns for Building LLM-based Systems Patterns: evals, RAG, guardrails, feedback. eugeneyan.com/writing/llm-patterns

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	Build the eval harness on your 30-question golden set; extend to 50.
Tue	Retrieval metrics and failure analysis (look at 20 failures).
Wed	Generation evals: faithfulness and an LLM judge; check judge against your own labels.
Thu	Add tracing (Langfuse or Phoenix); record cost and latency.
Fri	Run 3 improvement experiments (chunking, reranking, prompt); log a before/after table.
Sat	Build UI, deploy publicly, write the eval report. (5–6 hrs)
Sun	README and write-up; retro; send first 5 applications; 4 DSA problems.

HANDS-ON PRACTICE

1. Eval harness with at least 50 questions and automated scoring.
2. Three controlled experiments with a before/after table (change one variable each).
3. Trace and categorise 10 failures: retrieval miss, bad chunk, hallucination, prompt issue.
4. Deploy a public demo with rate limiting and a clear "sources" panel.

DELIVERABLE (push to GitHub)

- PROJECT 2: Production-style RAG assistant: live demo URL, eval report (before/after metrics), architecture diagram, README, cost and latency notes
- Blog post or LinkedIn write-up of what improved the metrics

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Explain recall@k vs faithfulness and why you need both.
- Show evidence that a change improved your metrics, not just your impression.
- Describe the limitations of LLM-as-judge and how you mitigate them.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1-5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

10

Agents I: Tool Use, Planning and LangGraph

Phase 4 · Agents and Production

Dec 7 – Dec 13

~30 hrs

GOAL Understand the agent loop deeply, build it from scratch without a framework, then rebuild it with LangGraph.

WHAT YOU WILL LEARN

- Workflows vs agents, and when not to build an agent
- The agent loop: reason, act, observe; ReAct, planning, reflection
- Tool design: clear names, narrow schemas, helpful error messages
- Memory: short-term (state), long-term (retrieval), summarisation
- LangGraph: nodes, edges, state, checkpointing, human-in-the-loop
- Control: max steps, timeouts, cost caps, loop detection, structured outputs for routing
- Frameworks overview: smolagents, OpenAI Agents SDK, Google ADK, CrewAI

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Anthropic: Building Effective Agents Read fully. Note the workflow patterns: chaining, routing, parallelisation, orchestrator-workers, evaluator-optimizer. www.anthropic.com/engineering/building-effective-agents
CORE	Hugging Face Agents Course (free, certificate) Units 1–3: agent fundamentals, smolagents, LlamaIndex / LangGraph. huggingface.co/learn/agents-course
CORE	DeepLearning.AI Short Courses (free) Take "AI Agents in LangGraph" (free short course). www.deeplearning.ai/short-courses
CORE	LangChain Academy (free courses, incl. Intro to LangGraph) Take Introduction to LangGraph (free). academy.langchain.com
CORE	LangGraph documentation State, persistence and human-in-the-loop guides. langchain-ai.github.io/langgraph
OPTIONAL	Lilian Weng: LLM Powered Autonomous Agents Conceptual overview of planning, memory and tool use. lilianweng.github.io/posts/2023-06-23-agent
OPTIONAL	smolagents documentation Code-writing agents; compare with tool-calling agents. huggingface.co/docs/smolagents
OPTIONAL	OpenAI Agents SDK documentation Another lightweight agent framework. openai.github.io/openai-agents-python
OPTIONAL	Microsoft: AI Agents for Beginners Lessons on agentic design patterns. github.com/microsoft/ai-agents-for-beginners

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	Read Building Effective Agents; write a 100-line agent loop with raw API calls.
Tue	Tool design: build 3 tools with strict schemas and clear errors.
Wed	LangGraph basics: rebuild the loop as a graph.
Thu	State, memory, checkpoints, human-in-the-loop approval step.
Fri	HF Agents course units; compare smolagents to your versions.
Sat	Build the research assistant agent and 20-task test set. (5–6 hrs)
Sun	Failure log, retro, 4 DSA problems (DP basics), plan Week 11.

HANDS-ON PRACTICE

1. Raw agent loop (no framework) with tool calling and a max-steps guard.
2. Same agent in LangGraph with persistence and an approval step for risky tools.
3. Research assistant with search, calculator and file-reader tools; log every step.
4. Create 20 test tasks; report success rate, average steps and cost per task.

DELIVERABLE (push to GitHub)

- Agent repo with both implementations, test tasks, success-rate table and a failure log

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Decide workflow vs agent for three scenarios and justify.
- Explain ReAct and how you stop infinite loops.
- Explain LangGraph state and checkpointing.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1-5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

WEEK
11

Agents II: MCP, Safety and Capstone Build

Phase 4 · Agents and Production

Dec 14 – Dec 20
~30 hrs

GOAL Add MCP, guardrails and agent evaluation, and build the capstone MVP end to end.

WHAT YOU WILL LEARN

- Model Context Protocol: hosts, clients, servers; tools, resources, prompts
- Building an MCP server in Python and connecting an agent to it
- Multi-agent patterns: orchestrator-worker, router, evaluator-optimizer; cost of coordination
- Security: prompt injection, least-privilege tools, input/output validation, PII handling, sandboxing
- Human approval for risky actions; audit logs
- Agent evaluation: task success, trajectory quality, tool-call accuracy

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Model Context Protocol documentation Introduction, architecture, and the "build a server" tutorial. modelcontextprotocol.io
CORE	Hugging Face MCP Course Work through the units; build and consume a server. huggingface.co/learn/mcp-course
CORE	DeepLearning.AI Short Courses (free) Take "MCP: Build Rich-Context AI Apps with Anthropic" (free). www.deeplearning.ai/short-courses
CORE	MCP Python SDK Use the Python SDK to expose your own tools. github.com/modelcontextprotocol/python-sdk
CORE	OWASP Top 10 for LLM Applications Read the top 10: prompt injection, insecure output handling, excessive agency. genai.owasp.org/llm-top-10
OPTIONAL	Reference MCP servers Read reference servers for good tool design. github.com/modelcontextprotocol/servers
OPTIONAL	CrewAI documentation Multi-agent framework; compare with your own orchestration. docs.crewai.com
OPTIONAL	Google Agent Development Kit (ADK) Another agent framework with built-in eval tooling. google.github.io/adk-docs

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	MCP concepts; run an existing server and call it from a client.
Tue	Write your own MCP server with 3–5 real tools.
Wed	Connect your agent to the server; add tool-level permissions.
Thu	Red-team: 10 prompt-injection attempts; write mitigations and tests.
Fri	Agent eval set (30 tasks) with success and trajectory scoring.
Sat	Capstone MVP working end to end locally. (5–6 hrs)
Sun	Demo to a friend, collect feedback; retro; 4 DSA problems; plan Week 12.

HANDS-ON PRACTICE

1. MCP server exposing 3–5 useful tools (notes search, task manager, repo summariser, or domain data).
2. Write 10 injection attacks (in documents, tool outputs, user input) and show how your agent defends.
3. Build a 30-task agent evaluation set and report scores.
4. Start the capstone: RAG + agent + MCP + eval + simple web UI.

DELIVERABLE (push to GitHub)

- Capstone MVP running end to end locally
- MCP server repo and an injection-test report

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Explain MCP architecture in two minutes with a diagram.
- List five LLM-app security risks and a mitigation for each.
- Argue when multi-agent is worth the extra cost and when it is not.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1–5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

12

Production: Deployment, LLMOps and Fine-tuning

Phase 4 · Agents and Production

Dec 21 – Dec 27
~30 hrs

GOAL Ship the capstone as a real service and learn to operate it. Understand fine-tuning (LoRA / QLoRA) and when to use it instead of prompting or RAG.

WHAT YOU WILL LEARN

- FastAPI service design: async, streaming responses, auth, rate limiting
- Docker: images, containers, compose; configuration and secrets
- CI/CD with GitHub Actions: run tests and a small eval suite on every pull request
- Free-tier deployment: Hugging Face Spaces, Render, Google Cloud Run; Vercel for a frontend
- Monitoring: logs, traces, cost dashboards; caching, batching; provider fallback
- Fine-tuning: SFT, LoRA / QLoRA, dataset preparation, when not to fine-tune
- Local inference and quantisation overview (Ollama, llama.cpp)

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	FastAPI: Deployment guide Deployment concepts, workers, Docker. fastapi.tiangolo.com/deployment
CORE	Docker: Get started Write a Dockerfile and docker-compose.yml for the capstone. docs.docker.com/get-started
CORE	GitHub Actions documentation Workflow that installs, tests and runs a smoke eval. docs.github.com/en/actions
CORE	Hugging Face Spaces documentation Deploy a Docker or Gradio / Streamlit Space. huggingface.co/docs/hub/spaces
CORE	Hugging Face smol course (fine-tuning and alignment) Supervised fine-tuning module; run in Colab. github.com/huggingface/smol-course
CORE	Hugging Face PEFT documentation (LoRA) LoRA concepts and the PEFT API. huggingface.co/docs/peft
OPTIONAL	Hugging Face TRL documentation (SFT, DPO) SFTTrainer and preference optimisation. huggingface.co/docs/trl
OPTIONAL	Unsloth documentation (free Colab fine-tuning notebooks) Free Colab notebooks for fast LoRA fine-tuning. docs.unsloth.ai
OPTIONAL	Made With ML (MLOps course) Reference for testing, CI/CD and monitoring. madewithml.com
OPTIONAL	MLOps Zoomcamp (DataTalks.Club) Deployment and monitoring modules. github.com/DataTalksClub/mlops-zoomcamp

PRIORITY	RESOURCE, WHAT TO DO AND LINK
OPTIONAL	Full Stack LLM Bootcamp LLMOps lectures: evals, deployment, UX. fullstackdeeplearning.com/llm-bootcamp
OPTIONAL	Render documentation (free web service tier) Free web service deployment. render.com/docs
OPTIONAL	Google Cloud Run quickstarts Container deployment with a free tier. cloud.google.com/run/docs/quickstarts

DAY-BY-DAY PLAN

WHEN	FOCUS
Mon	Wrap the capstone in FastAPI: streaming endpoint, API-key auth, rate limit.
Tue	Dockerise; docker-compose for app + vector store.
Wed	GitHub Actions: tests + smoke eval on pull request; README badge.
Thu	Deploy; add monitoring, cost log, provider fallback.
Fri	LoRA fine-tuning lab on Colab; compare with a prompting baseline.
Sat	Capstone polish, architecture diagram, 2-3 minute demo video. (5-6 hrs)
Sun	Retro; 4 DSA problems; first mock interview; plan the final sprint.

HANDS-ON PRACTICE

1. Dockerise the capstone and run it with a single command.
2. CI: tests + a 10-question smoke eval must pass before merge.
3. Deploy publicly with API-key auth and rate limiting; add provider fallback.
4. Fine-tune a small model with LoRA on a narrow task; compare against the best prompt baseline with numbers.

DELIVERABLE (push to GitHub)

- PROJECT 3 (capstone) deployed: live URL, Docker image, CI badge, eval results, architecture diagram, README, 2-3 minute demo video
- Fine-tuning notebook with a baseline comparison

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Explain image vs container, and what your CI pipeline runs.
- Decide between prompting, RAG and fine-tuning for three scenarios.
- Describe how you would monitor quality, cost and latency in production.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / 30 target Energy (1-5): _____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

13

Final Sprint: Portfolio, Resume and Interviews

Phase 5 · Launch

Dec 28 – Dec 31
~16 hrs

GOAL Turn twelve weeks of work into interviews and offers: a polished portfolio, a sharp resume, mock interviews and an application pipeline.

WHAT YOU WILL LEARN

- Portfolio storytelling: problem, approach, results, trade-offs
- Resume bullets with measurable results; LinkedIn and GitHub profile
- Technical interview loop for AI engineers: coding, ML fundamentals, LLM system design, behavioural
- Application pipeline: tailoring, referrals, follow-ups, tracking

RESOURCES (FREE)

PRIORITY	RESOURCE, WHAT TO DO AND LINK
CORE	Chip Huyen: Introduction to Machine Learning Interviews (free book) Use the question bank and interview-loop overview. huyenchip.com/ml-interviews-book
CORE	Latent Space: The Rise of the AI Engineer Understand how the AI engineer role is defined and valued. www.latent.space/p/ai-engineer
CORE	NeetCode roadmap (DSA) Revise your weakest 3 DSA patterns. neetcode.io/roadmap
OPTIONAL	Chip Huyen: blog (AI systems, agents, evals) AI systems design material for interview prep. huyenchip.com/blog
OPTIONAL	Overleaf resume templates (LaTeX) ATS-friendly LaTeX templates. www.overleaf.com/latex/templates/tagged/cv

DAY-BY-DAY PLAN

WHEN	FOCUS
Day 1 · Mon Dec 28	Portfolio polish: pin 3 projects, README with demo GIF, architecture diagram, results table.
Day 2 · Tue Dec 29	Resume v1, LinkedIn headline and About, GitHub profile README; ask 3 people for a review.
Day 3 · Wed Dec 30	Two mock interviews (technical + AI system design); fix the top 3 gaps.
Day 4 · Thu Dec 31	Re-score the rubric (Section 11), apply to at least 10 roles, publish a "what I built in 90 days" post, plan the next 90 days.

HANDS-ON PRACTICE

1. Record yourself explaining each project in 2 minutes; cut rambling.
2. Prepare 5 STAR stories: failure, conflict, leadership, ambiguity, a technical decision.
3. Whiteboard two AI system designs from the question list (Section 7).
4. Send at least 10 tailored applications and 5 referral requests.

DELIVERABLE (push to GitHub)

- Portfolio ready: 3 pinned projects with live demos, READMEs, eval results
- Resume v1, LinkedIn updated, 10 applications sent, final rubric scores, next-90-days plan

YOU ARE READY TO MOVE ON WHEN YOU CAN

- Explain each project in two minutes: problem, design, results, trade-offs, what you would change.
- Answer: "How do you know your AI feature works?" with evals, traces and numbers.
- Walk through one RAG and one agent system design on a whiteboard.

WEEKLY RETRO (20 MINUTES, LAST DAY OF THE WEEK)

Hours logged: _____ / — target Energy (1–5): ____
Wins this week:
Blockers and what I will do about them:
Top 3 outcomes for next week:
☐ Code pushed ☐ Deliverable done ☐ Notes written ☐ 4 DSA problems

6. Portfolio and Capstone Projects

Three projects tell a clear story: **you can model data** (Project 1), **you can build and evaluate retrieval systems** (Project 2) and **you can build safe, deployed agents** (Project 3). Quality beats quantity: a small project with an eval report and a live demo outperforms five unfinished notebooks.

PROJECT	WHEN	MUST HAVE	STRETCH GOALS
1. ML service Tabular prediction with a served API	Week 4	Real dataset and business question; leakage-safe Pipeline; 3+ models with cross-validation; MLflow results table; FastAPI /predict with tests; model card	SHAP explanation endpoint; Docker image; drift check script
2. RAG assistant Production-style retrieval with evals	Week 9	Real corpus (50+ documents); hybrid search + reranking; answers with citations; golden set of 50+ questions; before/after eval table; tracing; live demo; cost and latency notes	Contextual retrieval; query rewriting; user feedback buttons; per-language evals
3. Agentic capstone RAG + agent + MCP + guardrails	Weeks 10 – 13	Agent loop (LangGraph) with 3+ tools; custom MCP server; guardrails and injection tests; 30-task eval set; FastAPI + Docker + CI; deployed URL; architecture diagram; 2–3 minute demo video	Human approval step; provider fallback; fine-tuned small model comparison; usage dashboard

CAPSTONE IDEAS (PICK ONE YOU CAN TALK ABOUT WITH ENERGY)

IDEA	WHAT IT DOES	WHY IT IMPRESSES
Docs and codebase assistant	RAG over a library or repo plus MCP tools to search issues and summarise pull requests	Directly relevant to developer-tool companies
Research assistant	Plans a question, searches, reads sources, writes a cited brief; human approval before final report	Shows planning, tools, memory and citation quality
Support triage agent	Classifies tickets, retrieves policy, drafts a reply, escalates when unsure	Maps to real business workflows; easy to evaluate
Multilingual document assistant	Answers questions over documents in English and Hindi, with separate evals per language	Shows evaluation skill and a real-world differentiator

DEFINITION OF DONE (EVERY PROJECT)

- A stranger can run it in 5 minutes from the README
- Tests pass in CI on every pull request
- Live demo link or recorded GIF at the top of the README
- Architecture diagram and the key design trade-offs
- Evaluation results with numbers, not adjectives
- Known limitations and failure cases documented
- Cost and latency per request recorded
- No secrets in the repo; .env.example provided

README TEMPLATE

1. One-line pitch and demo link 2. Problem and users 3. Architecture diagram 4. Tech stack and why 5. Results table (metrics before and after) 6. How to run (3 commands) 7. Evaluation method 8. Limitations and security notes 9. What I would do next.

7. Interview Preparation

Plan for a loop with several rounds. Preparing each one in small daily doses beats cramming in the last week. Start light in Week 1 and increase from Week 9.

ROUND	WHAT THEY TEST	HOW YOU PREPARE
Online assessment / coding	DSA basics, SQL, Python fluency	4 problems per week from Week 0; NeetCode patterns; LeetCode SQL
Technical fundamentals	ML basics, Python, CS core (OOP, DBMS, OS, networks)	Question banks below; 1 hour per week on CS core
Project deep-dive	What you built, why, trade-offs, results	Two-minute explanation of each project; know your numbers
AI / LLM system design	RAG, agents, evals, cost, safety, scaling	Design prompts below; practise on a whiteboard from Week 9
Behavioural	Ownership, teamwork, learning, communication	Five STAR stories; practise aloud

DATA STRUCTURES AND ALGORITHMS TRACK

Keep DSA at a steady **4 problems per week** (30–40 minutes a day, four days a week). Follow patterns, not random problems. Use neetcode.io/roadmap, leetcode.com and takeuforward.org.

WEEKS	PATTERNS
0 – 2	Arrays and hashing, two pointers, sliding window
3 – 5	Stack, binary search, linked lists, trees, heaps, recursion
6 – 8	Graphs (BFS / DFS), intervals, backtracking, tries
9 – 13	Dynamic programming basics, review of weakest patterns, timed mock rounds (45 minutes per two problems)

ML FUNDAMENTALS QUESTION BANK (BE ABLE TO ANSWER IN 60–90 SECONDS)

- Explain the bias-variance trade-off with an example.
- How do you detect and reduce overfitting?
- When do you use L1 vs L2 regularisation?
- Precision vs recall: when does each matter? What is F1?
- ROC-AUC vs PR-AUC for imbalanced data?
- What is cross-validation and when does it fail (time series, groups)?
- What is data leakage? Give two real examples.
- Random forest vs gradient boosting: differences and trade-offs.
- Explain gradient descent, learning rate and mini-batches.
- Why do deep networks suffer from vanishing gradients and how do we mitigate it?
- What does batch normalisation do? What does dropout do?
- How does PCA work and what are its limits?
- How do you handle missing values and categorical variables?
- How do you evaluate a regression model and a ranking model?
- What is model calibration and why does it matter?
- How would you handle a heavily imbalanced dataset?
- What is an embedding and how is it learned?
- Why does feature scaling matter for some models and not others?

LLM AND GENAI QUESTION BANK

- Explain self-attention and why transformers replaced RNNs.
- What is tokenisation (BPE) and how does it affect cost and behaviour?
- What do temperature, top-k and top-p do?
- Why do LLMs hallucinate? List mitigations.
- RAG vs fine-tuning vs prompting: how do you choose?
- How do you choose chunk size and overlap?
- Dense vs sparse vs hybrid retrieval; what does a reranker add?
- How do you evaluate a RAG system end to end?
- What are the limits of LLM-as-judge and how do you calibrate it?
- How does function / tool calling work under the hood?
- Workflow vs agent: when is an agent the wrong choice?
- Describe ReAct and common agent failure modes.
- What is MCP and what problem does it solve?
- What is prompt injection? How do you defend against it?
- How would you cut LLM cost and latency by 50%?
- How do caching, batching and streaming help in production?
- What are LoRA and QLoRA, and when would you use them?
- What is the KV cache and why does context length cost memory?
- RLHF vs DPO in one paragraph.
- How do you guarantee valid structured output from a model?
- How do you version and test prompts?
- What would you monitor in a production LLM application?

AI SYSTEM DESIGN PROMPTS (30–40 MINUTES EACH)

1. Design a customer-support assistant over 1 million documents that must cite sources and avoid wrong answers.
2. Design an agent that books travel by calling external APIs, with approval for payments.
3. Design an evaluation pipeline that runs on every pull request for a RAG product.
4. Design semantic search for an e-commerce catalogue in multiple languages.
5. Your LLM feature costs too much and responds too slowly. Walk through diagnosis and fixes.
6. Design guardrails for an agent that can read email and call internal tools.
7. Design a document-ingestion pipeline for PDFs with tables and scanned pages.
8. How would you monitor quality drift after launch and decide when to retrain, re-index or re-prompt?

MOCK INTERVIEW SCHEDULE

WHEN	FORMAT	WITH
Week 9 (Dec 1–6)	Project walk-through + 1 coding problem	A friend or senior
Week 11 (Dec 14–20)	ML + LLM fundamentals round	A peer who is also preparing
Week 12 (Dec 21–27)	AI system design round	A mentor, senior or an online peer-mock community
Week 13 (Dec 29–30)	Two full mock rounds; record and review them	Two different people

8. Portfolio, Résumé and Job Hunt

GITHUB CHECKLIST

- Profile README: who you are, what you are building, links to demos
- Three pinned repositories (the three projects)
- Clear commit history with real messages
- Topics and a short description on each repo
- README with demo GIF at the top
- License and .env.example in each project

RÉSUMÉ: ONE PAGE, ATS FRIENDLY

Order: **Education, Projects, Skills, Experience** (flip Experience up if it is strong). Each bullet uses the formula **action + what + how + measurable result**. Use numbers from your own evals.

WEAK BULLET	STRONG BULLET (example format: replace with your real numbers)
Made a chatbot using LangChain	Built a RAG assistant over 1,200 pages with hybrid search and reranking; raised recall@5 from 0.62 to 0.84 on a 60-question golden set; deployed on Hugging Face Spaces with CI evals.
Worked on an AI agent	Designed a LangGraph agent with a custom MCP server and approval step for risky tools; achieved 80% task success on a 30-task evaluation set and blocked 10 of 10 injection attempts.
Trained ML model	Built a churn-prediction service (XGBoost, scikit-learn Pipeline) with MLflow tracking and a FastAPI endpoint; improved F1 from 0.58 to 0.71 over a logistic-regression baseline.

Skills section: list tools you can discuss for five minutes: Python, FastAPI, PyTorch, scikit-learn, SQL, RAG, LangGraph, MCP, Docker, GitHub Actions, evaluation. Avoid keyword stuffing; every item must be defensible in an interview.

LINKEDIN AND BUILDING IN PUBLIC

- **Headline:** "AI Engineer | RAG, Agents, LLM Evals | Python, FastAPI" (adjust honestly to your level).
- **Featured:** your three project demos and your blog post.
- **Posting rhythm:** one short post per week: what you built, one lesson, one chart. It creates inbound opportunities.
- **Reading:** follow practitioners who share real evals and failure stories (see the blogs in Section 9).

WHERE TO APPLY AND WHAT TO SEARCH

CHANNEL	EXAMPLES	TIP
Campus placements	Your placement cell and premium tracks	Prepare for aptitude, DSA and CS core even if you target AI roles
Job boards	LinkedIn Jobs, Naukri, Wellfound, Instahyre, Cutshort, Internshala	Set alerts for the title keywords below; apply within 48 hours of posting
Startups and remote	Y Combinator Work at a Startup, company career pages, startup communities	Smaller teams value shipped projects over credentials
Referrals	Alumni, seniors, open-source maintainers, people whose posts you engage with	Send a short message with your project link and one specific question

Search keywords: AI Engineer, Generative AI Engineer, LLM Engineer, Applied AI Engineer, AI Software Engineer, Agentic AI Developer, ML Engineer, Python Developer (AI), AI Intern, Associate Software Engineer (AI).

COLD MESSAGE TEMPLATE

"Hi [Name], I am a final-year BTech student building AI applications. I recently shipped a RAG assistant with an evaluation report ([link]) and an agentic capstone ([link]). I am applying for [role] at [company] and would value a few minutes of your perspective on what the team looks for in junior AI engineers. Thank you for your time."

APPLICATION TRACKER (COPY INTO A SHEET)

Columns: Company · Role · Link · Date applied · Referral? · Resume version · Stage · Follow-up date · Notes. Target **10 tailored applications per week** from Week 9, and **5 referral requests per week**.

9. Master Resource Index

Every link used in this guide, grouped by topic. Titles are clickable in the PDF. Verify availability, since websites and courses change.

Setup, tooling and free compute / APIs

GitHub: Start your journey docs.github.com/en/get-started/start-your-journey	Pro Git (free book) git-scm.com/book/en/v2
uv: fast Python project and package manager docs.astral.sh/uv	Python in Visual Studio Code code.visualstudio.com/docs/python/python-tutorial
The Missing Semester of Your CS Education (MIT) missing.csail.mit.edu	Google Colab colab.research.google.com
Kaggle Notebooks www.kaggle.com/code	Hugging Face (models, datasets, Spaces) huggingface.co
Google AI Studio (free Gemini API key) aistudio.google.com	Groq Console (free-tier API key) console.groq.com
OpenRouter (free-tagged models) openrouter.ai	Ollama (run open models locally) ollama.com

Python and software engineering

The Python Tutorial (official docs) docs.python.org/3/tutorial	CS50's Intro to Programming with Python (Harvard) cs50.harvard.edu/python
Real Python (free articles) realpython.com	asyncio (official Python docs) docs.python.org/3/library/asyncio.html
HTTPX (async HTTP client) www.python-httpx.org	Pydantic documentation docs.pydantic.dev/latest
pytest: getting started docs.pytest.org/en/stable/getting-started.html	FastAPI tutorial fastapi.tiangolo.com/tutorial

Data skills and math

Kaggle Learn: Python www.kaggle.com/learn/python	Kaggle Learn: Pandas www.kaggle.com/learn/pandas
Kaggle Learn: Data Visualization www.kaggle.com/learn/data-visualization	NumPy quickstart numpy.org/doc/stable/user/quickstart.html
Python Data Science Handbook (free online) jakevdp.github.io/PythonDataScienceHandbook	Kaggle Learn: Intro to SQL www.kaggle.com/learn/intro-to-sql
Kaggle Learn: Advanced SQL www.kaggle.com/learn/advanced-sql	SQLBolt (interactive SQL lessons) sqlbolt.com
LeetCode: Top SQL 50 study plan leetcode.com/studyplan/top-sql-50	3Blue1Brown: Essence of Linear Algebra www.3blue1brown.com/topics/linear-algebra
3Blue1Brown: Essence of Calculus www.3blue1brown.com/topics/calculus	StatQuest with Josh Starmer (YouTube) www.youtube.com/@statquest
Khan Academy: Statistics and Probability www.khanacademy.org/math/statistics-probability	Mathematics for Machine Learning (free book) mml-book.github.io

Classical machine learning

Machine Learning Specialization, Andrew Ng (audit free) www.coursera.org/specializations/machine-learning-introduction	Kaggle Learn: Intro to Machine Learning www.kaggle.com/learn/intro-to-machine-learning
Kaggle Learn: Intermediate Machine Learning www.kaggle.com/learn/intermediate-machine-learning	Kaggle Learn: Feature Engineering www.kaggle.com/learn/feature-engineering

scikit-learn User Guide scikit-learn.org/stable/user_guide.html	Google Machine Learning Crash Course developers.google.com/machine-learning/crash-course
Hands-On Machine Learning notebooks (Aurelien Geron) github.com/ageron/handson-ml3	Machine Learning Zoomcamp (DataTalks.Club) github.com/DataTalksClub/machine-learning-zoomcamp
XGBoost documentation xgboost.readthedocs.io	MLflow documentation mlflow.org/docs/latest
SHAP documentation shap.readthedocs.io	Google's Rules of Machine Learning developers.google.com/machine-learning/guides/rules-of-ml
Kaggle competition: Titanic www.kaggle.com/c/titanic	Kaggle competition: House Prices www.kaggle.com/c/house-prices-advanced-regression-techniques
Deep learning and PyTorch	
Andrej Karpathy: Neural Networks, Zero to Hero karpathy.ai/zero-to-hero.html	PyTorch: Learn the Basics pytorch.org/tutorials/beginner/basics/intro.html
3Blue1Brown: Neural Networks www.3blue1brown.com/topics/neural-networks	fast.ai: Practical Deep Learning for Coders course.fast.ai
Dive into Deep Learning (free book) d2l.ai	Karpathy: A Recipe for Training Neural Networks karpathy.github.io/2019/04/25/recipe
Stanford CS231n course notes cs231n.github.io	
Transformers and LLM internals	
Hugging Face LLM Course huggingface.co/learn/llm-course	The Illustrated Transformer (Jay Alammar) jalammar.github.io/illustrated-transformer
The Illustrated GPT-2 (Jay Alammar) jalammar.github.io/illustrated-gpt2	Attention Is All You Need (paper) arxiv.org/abs/1706.03762
Build a Large Language Model From Scratch (code, Sebastian Raschka) github.com/rasbt/LLMs-from-scratch	Stanford CS224N: NLP with Deep Learning web.stanford.edu/class/cs224n
LLM Course roadmap (Maxime Labonne) github.com/mlabonne/llm-course	
LLM APIs, prompting and context engineering	
DeepLearning.AI Short Courses (free) www.deeplearning.ai/short-courses	Anthropic courses (interactive prompting and tool-use tutorials) github.com/anthropics/courses
Anthropic docs: Prompt engineering overview docs.claude.com/en/docs/build-with-claude/prompt-engineering/overview	Gemini API documentation ai.google.dev/gemini-api/docs
OpenAI Cookbook cookbook.openai.com	Prompt Engineering Guide www.promptingguide.ai
LiteLLM (one interface to many providers) docs.litellm.ai	Anthropic Cookbook github.com/anthropics/anthropic-cookbook
Simon Willison's blog (practical LLM engineering) simonwillison.net	Microsoft: Generative AI for Beginners github.com/microsoft/generative-ai-for-beginners
RAG and retrieval	
LLM Zoomcamp (DataTalks.Club) github.com/DataTalksClub/llm-zoomcamp	Chroma documentation docs.trychroma.com
Sentence-Transformers documentation www.sbert.net	LlamaIndex documentation docs.llamaindex.ai

Pinecone Learning Center (RAG, chunking, reranking) www.pinecone.io/learn	Anthropic: Introducing Contextual Retrieval www.anthropic.com/news/contextual-retrieval
MTEB embedding leaderboard huggingface.co/spaces/mteb/leaderboard	FAISS (vector similarity search library) github.com/facebookresearch/faiss
pgvector (vector search in Postgres) github.com/pgvector/pgvector	
Evaluation, observability and demos	
Hamel Husain: blog on LLM evals hamel.dev/blog	Ragas documentation docs.ragas.io
Langfuse documentation (tracing and evals) langfuse.com/docs	promptfoo documentation (evals and red-teaming) www.promptfoo.dev/docs/intro
Arize Phoenix documentation docs.arize.com/phoenix	Eugene Yan: Patterns for Building LLM-based Systems eugeneyan.com/writing/llm-patterns
Streamlit documentation docs.streamlit.io	Hugging Face Spaces documentation huggingface.co/docs/hub/spaces
Agents, MCP and AI safety	
Anthropic: Building Effective Agents www.anthropic.com/engineering/building-effective-agents	Hugging Face Agents Course (free, certificate) huggingface.co/learn/agents-course
LangChain Academy (free courses, incl. Intro to LangGraph) academy.langchain.com	LangGraph documentation langchain-ai.github.io/langgraph
Lilian Weng: LLM Powered Autonomous Agents lilianweng.github.io/posts/2023-06-23-agent	smolagents documentation huggingface.co/docs/smolagents
OpenAI Agents SDK documentation openai.github.io/openai-agents-python	Microsoft: AI Agents for Beginners github.com/microsoft/ai-agents-for-beginners
Model Context Protocol documentation modelcontextprotocol.io	Hugging Face MCP Course huggingface.co/learn/mcp-course
MCP Python SDK github.com/modelcontextprotocol/python-sdk	Reference MCP servers github.com/modelcontextprotocol/servers
OWASP Top 10 for LLM Applications genai.owasp.org/llm-top-10	CrewAI documentation docs.crewai.com
Google Agent Development Kit (ADK) google.github.io/adk-docs	
Deployment, MLOps and fine-tuning	
FastAPI: Deployment guide fastapi.tiangolo.com/deployment	Docker: Get started docs.docker.com/get-started
GitHub Actions documentation docs.github.com/en/actions	Render documentation (free web service tier) render.com/docs
Google Cloud Run quickstarts cloud.google.com/run/docs/quickstarts	Hugging Face smol course (fine-tuning and alignment) github.com/huggingface/smol-course
Hugging Face PEFT documentation (LoRA) huggingface.co/docs/peft	Hugging Face TRL documentation (SFT, DPO) huggingface.co/docs/trl
Unsloth documentation (free Colab fine-tuning notebooks) docs.unsloth.ai	Made With ML (MLOps course) madewithml.com
MLOps Zoomcamp (DataTalks.Club) github.com/DataTalksClub/mlops-zoomcamp	Full Stack LLM Bootcamp fullstackdeeplearning.com/llm-bootcamp

Interviews, career and optional tracks

NeetCode roadmap (DSA)neetcode.io/roadmap**LeetCode**leetcode.com**takeuforward (DSA sheets and explanations)**takeuforward.org**Chip Huyen: Introduction to Machine Learning Interviews (free book)**huyenchip.com/ml-interviews-book**Chip Huyen: blog (AI systems, agents, evals)**huyenchip.com/blog**Latent Space: The Rise of the AI Engineer**www.latent.space/p/ai-engineer**Overleaf resume templates (LaTeX)**www.overleaf.com/latex/templates/tagged/cv**Hugging Face Computer Vision Course**huggingface.co/learn/computer-vision-course**Hugging Face Audio Course**huggingface.co/learn/audio-course

10. Progress Tracker

Tick the box when the deliverable is pushed and the self-check is complete. Log your real hours; adjust the next week rather than hiding a bad one.

WK	DATES	DELIVERABLE	DONE	HOURS	NOTES
0	Oct 1 – Oct 4	Repo, API keys, baseline scores	☐		
1	Oct 5 – Oct 11	FastAPI service with tests; async caller	☐		
2	Oct 12 – Oct 18	EDA notebook; gradient descent from scratch	☐		
3	Oct 19 – Oct 25	Two Kaggle submissions; metrics cheat sheet	☐		
4	Oct 26 – Nov 1	PROJECT 1: end-to-end ML service	☐		
5	Nov 2 – Nov 8	micrograd + CIFAR-10 CNN	☐		
6	Nov 9 – Nov 15	Mini-GPT + blog post	☐		
7	Nov 16 – Nov 22	llm-toolkit + three mini-apps	☐		
8	Nov 23 – Nov 29	RAG pipeline with citations; golden set	☐		
9	Nov 30 – Dec 6	PROJECT 2: RAG assistant, eval report, live demo	☐		
10	Dec 7 – Dec 13	Agent from scratch + LangGraph version	☐		
11	Dec 14 – Dec 20	Capstone MVP + MCP server + injection tests	☐		
12	Dec 21 – Dec 27	PROJECT 3: capstone deployed with CI, video	☐		
13	Dec 28 – Dec 31	Portfolio, résumé, 10 applications	☐		

Streaks that matter: code every day (even 20 minutes), push at least 5 days a week, 4 DSA problems every week, one public post every week from Week 6.

11. Self-Assessment Rubric

Score yourself honestly in **Week 0**, again at **mid-point (Week 6, Nov 15)** and at the **end (Dec 31)**.

SCORE	MEANING
1	Heard of it; cannot do it
2	Followed a tutorial; cannot do it alone
3	Built it independently and can explain it. This is the job-ready bar.
4	Built it to a good standard with tests, evals and documentation
5	Could teach it, review others' work and make design trade-offs confidently

SKILL AREA	JOB-READY BAR (SCORE 3)	WK 0	WK 6	DEC 31
Python and software engineering	Typed, tested code; async; Git workflow; FastAPI service shipped			
Data and SQL	EDA in Pandas; joins, windows, CTEs without notes			
ML fundamentals	Correct validation, metrics and leakage control; ensembles; pipelines			
Deep learning (PyTorch)	Own training loop; can explain backprop; can debug training			
Transformers and LLM internals	Explain attention, tokenisation, sampling; trained a small GPT			
LLM APIs and prompting	Structured output, tool calling, retries, cost and latency control			
RAG	Hybrid retrieval, reranking, citations over a real corpus			
Evaluation and observability	Golden sets, retrieval and generation metrics, tracing, regression tests			
Agents and MCP	Agent loop, LangGraph, custom MCP server, 30-task eval set			
Deployment and LLMOps	Docker, CI, free-tier deployment, monitoring, fallback			
Security and safety	Prompt-injection tests, least-privilege tools, PII handling			
DSA and CS fundamentals	Medium problems in 30–40 minutes; core OOP, DBMS, OS, networks			
Communication and portfolio	READMEs, demos, 2-minute project explanations, résumé			

Target by Dec 31: at least **3** in every row and at least **4** in Python and software engineering, LLM APIs and prompting, RAG, and Evaluation and observability. Any row still at 2 becomes your first focus of the next 90 days.

12. If You Fall Behind (or Get Ahead)

Falling behind is normal. The goal is to protect the outcomes that get you hired, not to complete every video.

PROTECT THESE FIRST (NEVER CUT)

- The three deliverables, especially Project 2 (RAG with evals) and Project 3 (capstone deployed)
- Weeks 7 – 12: LLM APIs, RAG, evaluation, agents, MCP, deployment
- Evaluation work: golden sets, metrics, traces
- Pushing code and writing README files

CUT IN THIS ORDER

1. All OPTIONAL resources.
2. From-scratch exercises that duplicate understanding you already have (micrograd, mini-GPT). Keep the explanation, skip the extra variations.
3. Maths depth beyond the intuition videos and the checks in Week 2.
4. Kaggle second submission, extra DSA beyond 4 problems per week, extra framework comparisons.

COMPRESSED PATH: ABOUT 20 HOURS PER WEEK

WEEKS	DO	SKIP OR SHRINK
1 – 2	Python engineering, FastAPI, Pandas, SQL basics	Extended maths, optional courses
3 – 4	Kaggle Learn track only; build Project 1 with a simpler dataset	Ng course videos beyond the labs you need
5 – 6	PyTorch basics + one notebook; watch the Let's build GPT video without full coding	micrograd, extra CNN work
7 – 9	Full plan: these weeks are the core of the role	Optional frameworks; use plain Python where it is faster
10 – 12	Agent from scratch, LangGraph, one MCP server, deploy	Multi-agent frameworks and fine-tuning lab (read, do not run)

IF YOU ARE AHEAD

- Enter a Kaggle competition and publish a notebook; or build a second RAG project in a different domain.
- Contribute a documentation fix or small feature to an open-source AI library (look for "good first issue").
- Explore an optional track: speech and audio (Hugging Face Audio Course) or computer vision (Hugging Face Computer Vision Course), linked in Section 9.
- Fine-tune a larger open model with QLoRA and write up the cost, data and evaluation honestly.

AFTER DECEMBER 31: THE NEXT 90 DAYS

1. Apply daily and iterate on your résumé from interview feedback.
2. Ship one domain-specific project aligned with the companies you want.
3. Deepen evaluation, system design and one cloud platform.
4. Keep a weekly public post and one open-source contribution per month.

Final word. Consistency beats intensity. Ninety days of building, measuring and shipping will put you far ahead of people who only watch tutorials. Open the repo, make the next commit, and keep going.